RETAIL DATA AGENT · LANGCHAIN

How middleware works

Middleware is our own code that LangChain runs at fixed points inside the agent loop.
Think of it as hooks you plug into the loop without rewriting it.

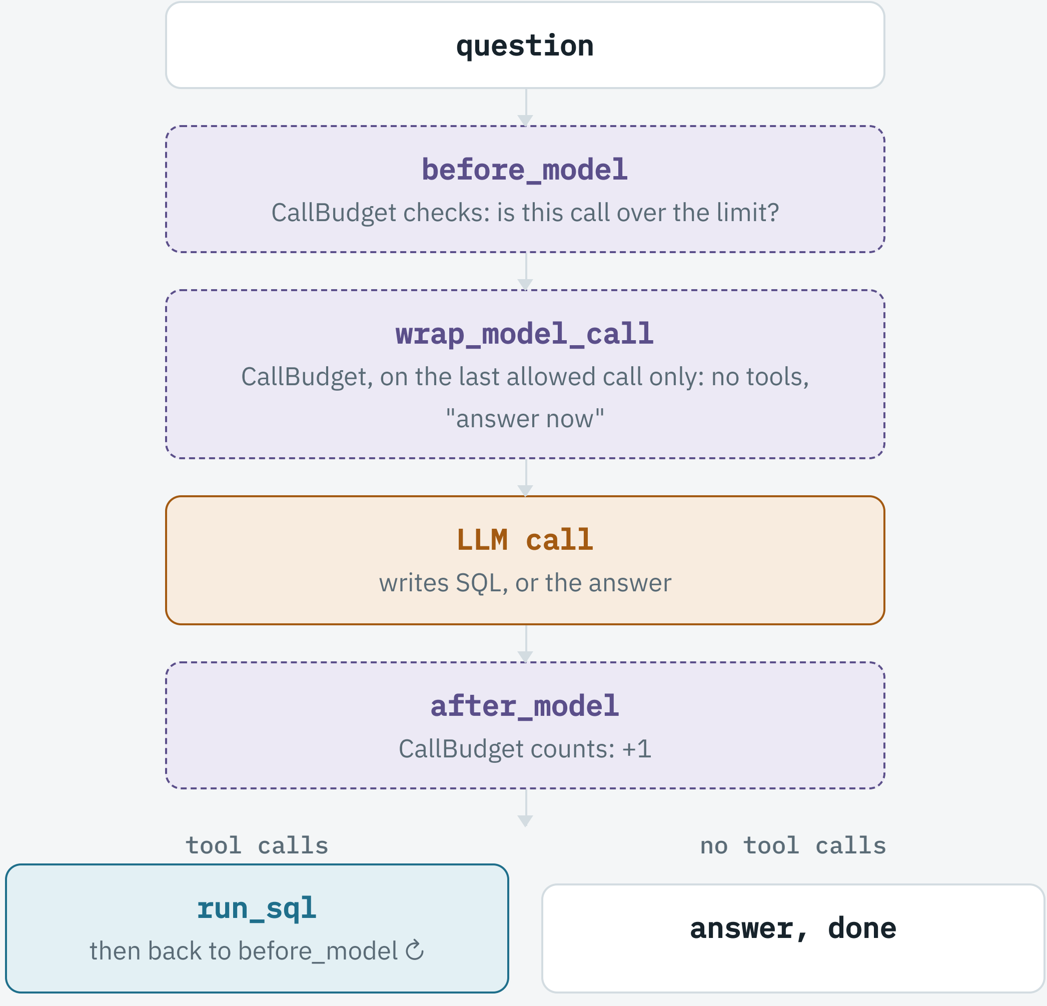
In short

`method name` decides **when** it runs: `before_model` runs before every LLM call, `after_model` after it.

`create_agent` does the **wiring**: it sees which hook methods a middleware overrides and plugs each one into the loop.

`@hook_config` decides **what it may do**, such as ending the loop early with `can_jump_to=["end"]`.

The agent loop, with the hooks


question
before_model
CallBudget checks: is this call over the limit?
wrap_model_call
CallBudget, on the last allowed call only: no tools, "answer now"
LLM call
writes SQL, or the answer
after_model
CallBudget counts: +1
tool calls
no tool calls
run_sql
then back to before_model ↻
answer, done

The dashed boxes are the hooks. Everything else is the normal loop that `create_agent` builds.

The hooks a middleware can use

LangChain's base class, `AgentMiddleware`, has a fixed set of hook methods. A middleware picks where it runs simply by overriding them.

METHOD	WHEN IT RUNS
`before_agent`	Once, when a question starts
`before_model`	Before every LLM call · **CallBudget checks the limit here**
`wrap_model_call`	Around every LLM call (can change the request or the response) · **CallBudget takes the tools away on the last call here**
`after_model`	After every LLM call · **CallBudget counts here**
`wrap_tool_call`	Around every tool call, such as `run_sql`
`after_agent`	Once, when the question ends

What `create_agent` does with it

1
Looks
Checks which hook methods each middleware overrides.

2
Wires
Plugs each overridden method into the loop at its fixed point, as a step in the LangGraph graph.

3
Skips
Leaves out the hooks it didn't override, so they cost nothing.

Nobody configures the position anywhere. `ModelCallLimitMiddleware` overrides `before_model` and `after_model`, and that alone puts it in the right place.

Example: how CallBudget ends a long question

It keeps a counter in the agent's state, reset for each new question. It never cuts a call off halfway. The 5th call gets no tools, so the model has to answer from the results it already fetched; a 6th call is never started.

LLM CALL	BEFORE_MODEL CHECK	RESULT
1	count 0 < 5 ✓	writes SQL, runs it
2	count 1 < 5 ✓	SQL failed, fixes it, runs it
3	count 2 < 5 ✓	another query
4	count 3 < 5 ✓	another query
5 (last)	count 4 < 5 ✓	`wrap_model_call` removes the tools and adds "answer now from the results you have, and say what is missing". The model answers.
6th attempt	count 5 → stop	Only if the 5th call asked for SQL anyway: jumps to the end with "I couldn't finish this question within 5 model calls..."

If the model answers before the 5th call, nothing changes and the user gets a normal answer. Every SQL retry needs an LLM call, so capping LLM calls caps retries too.

Ours vs LangChain's

LangChain's `ModelCallLimitMiddleware` has the counter and the stop logic. **Our `CallBudget`** adds two things on top. `wrap_model_call` turns the last allowed call into an answer: no tools, the earlier tool calls and results rewritten as plain text (Gemini was seen calling `run_sql` anyway when it could see them), and a note to answer now. `before_model` swaps LangChain's terse "Model call limits exceeded: run limit (5/5)" for a clear message, and tags it so `app.py` can log the stop. Because it overrides `before_model`, it repeats the permission to end the loop; without the decorator the jump wouldn't be allowed.

```
class CallBudget(ModelCallLimitMiddleware):
    def wrap_model_call(self, request, handler):  # name = runs around every LLM call
        if request.state.get("run_model_call_count", 0) == self.run_limit - 1:   # the last allowed call
            request = request.override(
                tools=[], tool_choice=None,
                messages=_tool_exchanges_as_text(request.messages),
                system_message=SystemMessage(f"{system_text}\n\n{LAST_CALL_NOTE}"),
            )
        return handler(request)

    @hook_config(can_jump_to=["end"])          # may end the loop early
    def before_model(self, state, runtime):    # name = runs before every LLM call
        update = super().before_model(state, runtime)
        if update is not None:                 # over the limit
            update["messages"] = [AIMessage("I couldn't finish ...", name=BUDGET_REACHED)]
        return update
```

Code: `CallBudget` in `src/retail_agent/agent.py`, limit set by `MAX_LLM_CALLS` (default 5). Tests: `tests/test_turn.py`. See also `docs/sql-errors.html` for which errors reach the model.